Convolution algebra and the exponential Lipschitz estimate

Claude, with Daniel Murfet

2026-09-07

Abstract

First half of Astra #10's rank 5 (local Lipschitz dependence of the amplitude coefficients on the double-series data). We record the linear algebra of the weighted convolution algebra $(\ell^1_\rho(\mathbb{N}^2), *)$: bilinearity of `conv`, the right unit, closure of weighted summability under sums and differences, and the triangle inequality for $\|\cdot\|_\rho$. On top of this, the telescoping bound $\|a^{*n} - a'^{*n}\|_\rho \le nM^{n-1}\|a - a'\|_\rho$ for $\|a\|_\rho, \|a'\|_\rho \le M$ and the Lipschitz estimate for the coefficient exponential $\|E_a(t) - E_{a'}(t)\|_\rho \le |t|e^{|t|M}\|a - a'\|_\rho$ (`wnorm_expCoeff_sub_le`). Zero `sorry`/`axiom`.

1 The things formalised

```
theorem conv_sub_left (f g h) :
    conv (fun k => f k - g k) h = fun k => conv f h k - conv g h k
theorem conv_sub_right (f g h) :
    conv f (fun k => g k - h k) = fun k => conv f g k - conv f h k
theorem conv_delta_right (f) : conv f delta = f
theorem wsummable_add / wsummable_sub ( ) (h  : 0    ) (f g) (hf hg) : WSummable   (f ± g)
theorem wnorm_add_le ( ) (h  : 0    ) (f g) (hf hg) :
    wnorm   (fun k => f k + g k)   wnorm   f + wnorm   g
theorem wnorm_convPow_sub_le ( ) (h  : 0 <  ) (a a') (ha ha') (M) (hM hM') (n) :
    WSummable   (fun k => convPow a n k - convPow a' n k)
    wnorm   (fun k => convPow a n k - convPow a' n k)
       if n = 0 then 0 else n * M ^ (n - 1) * wnorm   (fun k => a k - a' k)
theorem expDiffRow_hasSum (t M D) :
    HasSum (expDiffRow t M D) (|t| * Real.exp (|t| * M) * D)
theorem wnorm_expCoeff_sub_le ( ) (h  : 0 <  ) (a a') (ha ha') (ha0 ha0') (M) (hM hM')
    (t) :
    WSummable   (fun k => expCoeff a t k - expCoeff a' t k)
    wnorm   (fun k => expCoeff a t k - expCoeff a' t k)
       |t| * Real.exp (|t| * M) * wnorm   (fun k => a k - a' k)
```

2 Mathematics

The convolution $(f * g)_k = \sum_{a \le k} f_a g_{k-a}$ is bilinear, so

$$a^{*(n+1)} - a'^{*(n+1)} = (a - a') * a^{*n} + a' * (a^{*n} - a'^{*n}),$$

and the submultiplicativity $\|f * g\|_\rho \le \|f\|_\rho \|g\|_\rho$ of unit 117 gives by induction $\|a^{*n} - a'^{*n}\|_\rho \le nM^{n-1}\|a - a'\|_\rho$. No associativity or commutativity of $*$ is needed for this: the identity uses only linearity in each slot. For the exponential $E_a(t)_k = \sum_{n \le |k|} \frac{t^n}{n!} a^{*n}_k$ (with $a_{00} = 0$, so the sum is really over all n), the difference is $\sum_n \frac{t^n}{n!}(a^{*n} - a'^{*n})_k$, and the joint majorant $g(n,k) = \frac{|t|^n}{n!}|a^{*n}_k - a'^{*n}_k|\rho^{|k|}$ has row sums $\le \frac{|t|^n}{n!} n M^{n-1} D = |t|\frac{(|t|M)^{n-1}}{(n-1)!}D$ ($n \ge 1$), summing to $|t|e^{|t|M}D$ with $D = \|a - a'\|_\rho$. Tonelli on $\mathbb{N} \times \mathbb{N}^2$ then gives the weighted bound. This is exactly the estimate $\|e^a - e^{a'}\| \le |t|e^{|t|M}\|a - a'\|$ one has in any Banach algebra, specialised to the unital algebra of double power series with the ρ-weighted ℓ^1 norm.

3 Paper-facing meaning

The amplitude coefficients of §4.2 arise as $c(s) = y * E_{\bar{x}}(\beta s)$; the differentiability or even continuity of the Taylor-tree coefficients in the data (ξ, η) passes through a Lipschitz estimate for E. Astra #10 ranked this fifth (after the parameter integration, measurability and weighted-norm units, all landed). Unit 143 combines it with the scalar factor $e^{\beta s x_{00}}$ into the amplitude estimate proper.

4 Lean notes

- `HasSum.congr_fun` takes the equation in the direction new = old; the first attempt with `.symm` was rejected with a type mismatch. Then `(hasSum_nat_add_iff' 1).1` with `simpa [h0]` (killing $\sum_{i<1} f_i = f_0 = 0$) transfers the shifted series to the unshifted one; `convert` on a `HasSum` produced an unfillable `AddCommMonoid` extensionality goal and should be avoided here.
- The telescoping induction is cleanest stated for $n+1$ (`wnorm_convPow_succ_sub_le`) with the closed form $(n+1)M^n$; the $n = 0$ case is the separate identity $\delta - \delta = 0$, and the general lemma carries an `if n = 0` guard so the row bound stays a single expression.
- Only the `expCoeff_eq_tsum`-style rewrite (finite sum = full `tsum` using `convPow_eq_zero_of_lt`) is needed to compare with the row of the majorant; the rest is the unit-118 majorant argument with `summable_prod_of_nonneg` and `Equiv.prodComm.tsum_eq`.